

Supermemory

A real, installable Claude Code plugin that automates the exact compound-learning pattern already on the deck – CLAUDE.md's Learned Rules, but automatic and cross-session, built out of a SessionStart hook and a PostToolUse hook.

Everything today measured the cost of one session. This lab answers the question that's left over: what does it cost to re-explain the same codebase to a brand-new session tomorrow? You'll install a real memory plugin, fix a bug you've already fixed once, save what you learned, and then prove – in a session that has never seen this conversation – that the memory survived.

IN THIS LAB

You'll use **Supermemory**, a real third-party Claude Code plugin – not an Anthropic feature, not something this program depends on. It's one example of a pattern (persistent, cross-session memory) that today's deck already named. Everything runs in **self-hosted local mode**: free, no account, no data leaving your machine.

→ [Repo Linked](#)

ON THIS PAGE

- 01 The idea in one minute
- 02 How this connects to today
- 03 Set up Supermemory (~5 min)
- 04 Walkthrough – fix it, save it, prove it (~15 min)
- 05 More examples to try on your own (~10 min)
- 06 Honest limits – read this before you trust it with real work
- 07 Cheat sheet

01 The Idea in One Minute

Claude Code sessions are stateless by default – close one and start another, and everything you explained is gone. Supermemory adds two things on top of that:

- A **SessionStart hook** – before you type anything, it fetches memories relevant to what you're about to do and injects them into context automatically.
- A **PostToolUse hook** – every time you edit a file, write one, or run a command, it captures that action in the background so it can be recalled later.

Reasoned recall, not a dump: before each turn, Claude decides for itself whether searching memory would help. It isn't blindly injecting everything it has ever seen – only a handful of the most relevant memories make it into context on any given turn.

02 How This Connects to Today

This lab deliberately reuses what you already built earlier today, so you can feel the difference rather than take it on faith:

- **Demo 1 (SubAgents):** you saw cheap-model routing work inside one session. Here you'll see what happens once that session ends.
- **Demo 2 (Agent Teams):** the bug you fix in Step 1 below is the exact same PATCH bug from Demo 2. You're fixing it a second time on purpose.
- **Demo 3 (Dynamic Workflows):** a saved workflow persists a repeatable process across sessions. This lab persists knowledge the same way – two different things you can both keep.

03 Set Up Supermemory (~5 min)

You need **Node.js 18+** on your PATH – the plugin's hooks are Node scripts.

In Claude Code, install the plugin:

CLAUDE CODE

```
/plugin marketplace add supermemoryai/claude-supermemory  
/plugin install supermemory
```

Then point it at your local server (your shell profile, or just export for this terminal):

SHELL

```
export SUPERMEMORY_CC_API_KEY="sm_local_xxxxxxx" # the key npx printed above  
export SUPERMEMORY_API_URL="http://localhost:6767"
```

Restart Claude Code so it picks up the env vars. **Checkpoint:** if the plugin install and both exports succeeded without errors, you're ready for Step 1.

SKIP – IF ALREADY CLONED

If you have already cloned the repo, please feel free to skip it. Make sure to `cd` in the right folder before starting the Claude session.

TERMINAL

```
git clone https://github.com/nsr-19/acceler-lvt-labs.git
cd acceler-lvt-labs/lvt-day4-code-demo && npm install && npm test
```

4 Walkthrough — Fix It, Save It, Prove It (~15 min)

Step 1 — Fix the bug, and watch what gets captured

From the TaskFlow repo root:

PROMPT

Fix the PATCH truthiness bug in `src/routes/tasks.js` the same way we did earlier, and explain the fix in one sentence.

While it works, notice the `PostToolUse` hook logging every edit and command in the background, roughly like this:

CAPTURED (BACKGROUND)

```
Edited src/routes/tasks.js: "if (req.body[field])" → "if (field in req.body)"
Ran: npm test (SUCCESS)
```

Step 2 — Save the rule, not just the diff

The auto-capture above logs *what changed*. It doesn't capture *why*, or that it's a rule worth remembering. Do that explicitly:

PROMPT

```
/supermemory:save We fixed PATCH by switching truthiness checks to explicit
"in" checks. Rule for this codebase: never use `if (req.body[field])` to
decide whether to apply an update — false, 0, and "" are valid values.
```

WHY THIS STEP MATTERS

This is the difference between what gets logged automatically and what a human decides is worth keeping. Only one of the two survives if you don't say it out loud.

Step 3 – Start completely fresh, and ask cold

Exit Claude Code entirely (``/exit`` , or close the terminal). Then start a brand-new session – no ``-continue`` , no ``-resume`` :

TERMINAL

claude

Without reminding it of anything, ask:

PROMPT

What's the convention for copying request fields safely in this codebase?

Checkpoint: the answer should be correct, and you should not have had to explain anything. If you have `SUPERMEMORY_DEBUG=1` set, you can also see the injected-context block directly.

Try the explicit search path too, for contrast with the automatic injection you just saw:

PROMPT

/supermemory:search truthiness bug

Step 4 – Know the limits before you trust this

- Reasoned recall caps out at roughly 5 memories per turn by default – it is not "remembers everything."
- The hosted product costs money. What you just ran is the free, self-hosted path.
- This is a third-party plugin. It captures your Edit/Write/Bash activity by default – know what it's storing before you point it at anything sensitive.

05 More Examples to Try on Your Own (~10 min)

If Step 1's single bug isn't enough to build real confidence, the same repo has three more genuine issues – pick any or all of them and repeat the fix → save → fresh session → confirm loop.

Example A – the $O(n^2)$ email check

PROMPT

Look at `isEmailTaken` in `src/services/userService.js` – is there a performance problem? Fix it and add a regression test.

What you'll find: a redundant nested loop over the same array where a single scan would do.

SAVE IT

/supermemory:save Never nest two loops over the same collection to check membership — that's $O(n^2)$ for what a single `.find().some()` does in $O(n)$. Rule for this codebase: check `userService.js` and `taskService.js` for this pattern whenever reviewing store-lookup helpers.

Example B — 200 instead of 404

PROMPT

If I request a user id that doesn't exist, what does `GET /api/users/:id` actually return? Fix it so a missing user is a proper 404.

SAVE IT

/supermemory:save Route handlers must check a service's return value before responding — `getUser()/getTask()` return null/undefined for "not found," and the route is responsible for turning that into a 404, not a 200 with an empty body.

Example C — the comment that lies

This is the most useful one of the three — it's not a code rule, it's a trust rule.

PROMPT

Does `createUser` in `src/services/userService.js` actually store arbitrary fields the way its comment claims? Check the real behavior against the comment and fix whichever one is wrong.

SAVE IT

/supermemory:save Don't trust a comment's claim about behavior without checking the code — this repo had a comment describing a mass-assignment vulnerability that the actual code doesn't have. Comments rot; re-verify before repeating a claim, the same way this memory itself should get re-checked if the code changes.

WORTH SITTING WITH

That last save is a reminder not to trust memories blindly. The same discipline applies to what Supermemory itself stores — if the code changes and nobody updates the saved rule, the rule goes stale exactly like the comment did.

06 Honest Limits — Read This Before You Trust It With Real Work

- **Reasoned recall caps out:** only ~5 memories make it into context per turn by default. That's a deliberate design choice (the same tight-payload idea from Part 4), not a bug – but it does mean you can't assume everything you ever saved is available every time.
- **The hosted product costs money:** this lab uses the free, self-hosted path. Evaluate the hosted Pro tier's pricing and data-handling terms yourself before using it for real project memory.
- **It's a third-party plugin, not a Claude Code built-in:** it captures your Edit/Write/Bash activity by default. Use `SUPERMEMORY_SKIP_TOOLS` to exclude anything you don't want logged.
- **CLAUDE.md is still a real alternative:** versioned, reviewable in a pull request, scoped to one repo. Supermemory is automatic and crosses projects. Neither is a strict upgrade over the other – pick based on what your team actually needs.

07 Cheat Sheet

COMMAND	WHAT IT DOES
<code>npx supermemory local</code>	starts the free, self-hosted memory server on <code>:6767</code>
<code>/plugin install supermemory</code>	installs the Claude Code plugin
<code>/supermemory:save <text></code>	explicitly persists a decision or rule
<code>/supermemory:search <query></code>	explicitly searches saved memories
<code>SUPERMEMORY_DEBUG=1</code>	env var – shows the injected-context block
<code>SUPERMEMORY_SKIP_TOOLS</code>	env var – comma-separated tools to exclude from capture

OPTIONAL (TRY LATER)

Start the local memory server and leave it running in its own terminal for the rest of this lab:

TERMINAL

```
npx supermemory local
# prints something like:
# Supermemory running at http://localhost:6767
# API key: sm_local_xxxxxxx
```

WHEN TO USE LOCAL, NOT THE HOSTED PRODUCT

The hosted Supermemory API needs a paid Pro plan and an account. The self-hosted server above is MIT-licensed, free, and mints its own API key on first boot—no signup needed for this lab.

Self-Check Before You Move On

1. Did Step 3's fresh session answer correctly without you re-explaining anything?
2. Can you explain, in one sentence, the difference between what PostToolUse auto-captures and what /supermemory:save persists?
3. Ran at least one of Examples A/B/C and confirmed it survives a fresh session the same way Step 3 did?
4. Can you name one real limit of this tool without looking back at Section 06?